

# Trabajo Práctico 5

1. Se verificó en clase el programa del factorial de  $x > 0$ , con la especificación  $(x > 0, y = x!)$ .

a. Justificar por qué esta especificación no es correcta.

La especificación no es correcta porque si bien se puede cumplir que  $x > 0$  antes de iniciar el programa, éste mismo puede modificar el valor de  $x$  durante su ejecución, lo que podría llevar a que  $y \neq x!$  al final del programa.

El punto principal es que la  $x$  de la postcondición se refiere al ÚLTIMO valor que tomó  $x$  durante la ejecución del programa, y no al valor inicial que tenía.

Podría darse este caso, por ejemplo:

- Valor inicial:  $x = 3$  y  $3 > 0$  (la precondition es verdadera).
- Durante la ejecución, el programa podría modificar  $x$  a 2, luego a 1, y finalmente a 0.
- Al finalizar,  $y$  podría ser igual a 6, siendo  $6 = 3!$ , lo cual sería válido, pero como el valor final de  $x$  es 0, la postcondición  $y = x!$  no se cumple, ya que  $0! = 1$ .

b. Proponer una que sí lo sea y que además establezca que el valor de  $x$  al final sea el mismo que al comienzo.

Propongo la especificación  $(x = X \wedge X > 0, y = X! \wedge x = X)$ , donde:

- $X$  es una variable lógica que representa el valor inicial de  $x$ .
- Por más que  $x$  cambie de valor durante la ejecución del programa, el valor de la variable lógica  $X$  se mantiene fijo.
- Al finalizar el programa se chequea que a)  $y$  sea igual al factorial del valor inicial de  $x$  (es decir,  $X!$ ) y b) que  $x$  no haya cambiado de valor en absoluto, ya que se chequea que  $x$  sea igual a  $X$  que como dijimos es el valor inicial de  $x$ .
- Por lo tanto, bajo esta especificación, si el programa factorial modifica la variable  $x$  durante su ejecución de forma tal que al terminar la ejecución  $x$  es distinto a su valor inicial, la postcondición no se cumple. Sin embargo, si modifica  $x$  pero antes de terminar lo vuelve a modificar dándole el valor inicial, entonces la postcondición sí se cumple.

c. ¿Podría agregarse a la especificación que el valor de  $x$  no se altere a lo largo de todo el programa? Justificar.

No es posible agregar ni una precondition ni una postcondición que asegure que el valor de cualquier variable **nunca** se modifique en absoluto en toda la ejecución.

Esto se debe a que la Lógica de Hoare solo puede analizar el estado del programa **antes** de ejecutarlo y **después** de ejecutarlo, pero no puede garantizar nada de lo que pasa en el medio (durante la ejecución).

## 2. Asumiendo $\{p\}S\{q\}$ , indicar en cada caso si vale lo afirmado. Justificar las respuestas:

Para este ejercicio es útil representar al significado de la correctitud parcial  $\{p\}S\{q\}$  con una tabla de verdad que relacione el valor de  $p$ , el de  $q$ , y la terminación o no del programa  $S$ , que denoto  $r$ .

La correctitud parcial dice: "Si el estado inicial satisface  $p$  y el programa  $S$  termina, entonces el estado final satisface  $q$ ". Esto se puede expresar como la implicación lógica  $(p \wedge r) \rightarrow q$ .

| $p$ | $q$ | $r$ | $(p \wedge r)$ | $(p \wedge r) \rightarrow q$ |
|-----|-----|-----|----------------|------------------------------|
| $V$ | $V$ | $V$ | $V$            | $V$                          |
| $V$ | $V$ | $F$ | $F$            | $V$                          |
| $V$ | $F$ | $V$ | $V$            | $F$                          |
| $V$ | $F$ | $F$ | $F$            | $V$                          |
| $F$ | $V$ | $V$ | $F$            | $V$                          |
| $F$ | $V$ | $F$ | $F$            | $V$                          |
| $F$ | $F$ | $V$ | $F$            | $V$                          |
| $F$ | $F$ | $F$ | $F$            | $V$                          |

El único caso donde la correctitud parcial no se cumple es cuando  $p = V$ ,  $q = F$ , y  $r = V$ , es decir, cuando el estado inicial satisface  $p$ , el programa termina, pero el estado final no satisface  $q$ .

### a. Si $S$ termina en un estado que satisface $q$ , entonces su estado inicial satisface $p$ .

Tenemos  $q = V$ ,  $r = V$ . Las filas de la tabla que cumplen esto son la primera y la quinta. En la primera fila,  $p = V$  y la implicación es verdadera, pero en la quinta fila,  $p = F$  y la implicación también es verdadera. Por lo tanto, no es cierto que si  $S$  termina en un estado que satisface  $q$ , entonces su estado inicial satisface  $p$ , ya que puede pasar que  $S$  termine en un estado que satisface  $q$  pero el estado inicial no satisfaga  $p$ .

**No vale lo afirmado.**

**b. Si  $S$  termina en un estado que no satisface  $q$ , entonces su estado inicial no satisface  $p$ .**

Tenemos  $q = F$ ,  $r = V$ . Las filas de la tabla que cumplen esto son la tercera y la séptima. En la tercera fila,  $p = V$  y la implicación es falsa, pero en la séptima fila,  $p = F$  y la implicación es verdadera. Por lo tanto sí es cierto si  $S$  termina en un estado que no satisface  $q$ , entonces su estado inicial no satisface  $p$ .

**Vale lo afirmado.**

**c. Si  $S$  no termina, entonces su estado inicial no satisface  $p$ .**

Tenemos  $r = F$ . Mirando las filas 2, 4, 6 y 8, vemos que en la fila 2,  $p = V$  y la implicación es verdadera, por lo tanto no es cierto que si  $S$  no termina, entonces su estado inicial no satisface  $p$ , ya que puede pasar que  $S$  no termine pero el estado inicial satisfaga  $p$ .

**No vale lo afirmado.**

**d. ¿Las respuestas en (a), (b) y (c) son las mismas considerando  $\langle p \rangle S \langle q \rangle$ ?**

$\langle p \rangle S \langle q \rangle$  se refiere a la correctitud total, que dice "Si el estado inicial satisface  $p$ , entonces el programa  $S$  termina y el estado final satisface  $q$ ".

Simbólicamente:  $p \rightarrow (r \wedge q)$ .

Reescribiendo la tabla de verdad:

| $p$ | $q$ | $r$ | $(r \wedge q)$ | $p \rightarrow (r \wedge q)$ |
|-----|-----|-----|----------------|------------------------------|
| $V$ | $V$ | $V$ | $V$            | $V$                          |
| $V$ | $V$ | $F$ | $F$            | $F$                          |
| $V$ | $F$ | $V$ | $F$            | $F$                          |
| $V$ | $F$ | $F$ | $F$            | $F$                          |
| $F$ | $V$ | $V$ | $V$            | $V$                          |
| $F$ | $V$ | $F$ | $F$            | $V$                          |
| $F$ | $F$ | $V$ | $F$            | $V$                          |
| $F$ | $F$ | $F$ | $F$            | $V$                          |

1. (a) Si  $S$  termina en un estado que satisface  $q$ , entonces su estado inicial satisface  $p$ .

i. Tenemos  $q = V$ ,  $r = V$ . Filas 1 y 5. Lo afirmado no se cumple porque puede pasar que  $S$  termine en un estado que satisface  $q$  pero el estado inicial no satisfaga  $p$  (fila 5).

ii. **No vale lo afirmado.**

2. (b) Si  $S$  termina en un estado que no satisface  $q$ , entonces su estado inicial no satisface  $p$ .

i. Tenemos  $q = F, r = V$ . Filas 3 y 7. Lo afirmado se cumple porque si  $S$  termina en un estado que no satisface  $q$ , entonces el estado inicial no satisface  $p$  (fila 7).

ii. **Vale lo afirmado.**

3. (c) Si  $S$  no termina, entonces su estado inicial no satisface  $p$ .

i. Tenemos  $r = F$ . Filas 2, 4, 6 y 8. Viendo las filas 6 y 8 que son las que cumplen la implicación, vemos que en ambos casos el estado inicial no satisface  $p$ , por lo tanto lo afirmado se cumple, a diferencia de la correctitud parcial, donde no se cumple.

ii. **Vale lo afirmado.**

### 3. Indicar en cada caso si vale lo afirmado. Justificar las respuestas:

**a. Se cumple  $\{x = 0\} \text{while } z = 0 \text{ do } z := 0 \text{ od } \{x = 0\}$ .**

1. Si al iniciar el programa se cumple  $x = 0$  y  $z = 0$  se entra en loop infinito. Tenemos  $p = V$  y  $r = F$ . Filas 2 y 4 de la tabla de correctitud parcial. En ambas filas la implicación es verdadera ya sea que  $q$  sea  $V$  o  $F$ .
2. Si al iniciar el programa se cumple  $x = 0$  y  $z \neq 0$  no se entra en loop infinito. Como el programa termina, se puede evaluar  $q$  al finalizar el programa, y como el programa no modifica  $x$ , entonces  $q$  se cumple. Tenemos  $p = V, r = V$ , y  $q = V$ . Fila 1 de la tabla de correctitud parcial, donde la implicación es verdadera.
3. Si al iniciar el programa se cumple  $x \neq 0$  y  $z = 0$  se entra en loop infinito. Tenemos  $p = F$  y  $r = F$ . Filas 6 y 8 de la tabla de correctitud parcial. En ambas filas la implicación es verdadera ya sea que  $q$  sea  $V$  o  $F$ .
4. Si al iniciar el programa se cumple  $x \neq 0$  y  $z \neq 0$  no se entra en loop infinito. Como el programa termina, se puede evaluar  $q$  al finalizar el programa, y como el programa no modifica  $x$ , entonces  $q$  no se cumple. Tenemos  $p = F, r = V$ , y  $q = F$ . Fila 7 de la tabla de correctitud parcial, donde la implicación es verdadera.

Como es imposible que la implicación de la C.P sea falsa, vale lo afirmado.

**b. Se cumple  $\langle x = 0 \rangle \text{while } z = 0 \text{ do } z := 0 \text{ od } \langle x = 0 \rangle$ .**

1. Si al iniciar el programa se cumple  $x = 0$  y  $z = 0$  se entra en loop infinito. Tenemos  $p = V$  y  $r = F$ . Filas 2 y 4 de la tabla de correctitud total. En ambas filas la implicación es falsa ya sea que  $q$  sea  $V$  o  $F$ .

Como se encontró un caso donde la implicación de la C.T es falsa, no vale lo afirmado.

**c. Si se cumple  $\{p_1 \wedge p_2\} S \{q_1 \wedge q_2\}$ , entonces  $\{p_1\} S \{q_1\}$  o bien  $\{p_2\} S \{q_2\}$ .**

Contraejemplo:

1. Sea  $S$  el programa que no hace nada (es solo una sentencia `skip`).
2. Sea  $p_1$  la precondition  $x = 0$ , y  $p_2$  la precondition  $x = 1$ .
3. Sean  $q_1 = F$  y  $q_2 = F$ .

4. Claramente  $(p_1 \wedge p_2) = F$  porque  $x$  no puede valer 0 y 1 al mismo tiempo.
5. Obviamente  $(q_1 \wedge q_2) = F$  porque ambos son falsos.
6. Entonces tenemos  $\{F\}S\{F\}$ , lo cual es válido porque si la precondition de una implicación es falsa, la implicación es verdadera.
7. Sin embargo, si  $p_1$  es  $x = 0$ , entonces  $\{p_1\}S\{q_1\}$  es  $\{x = 0\}S\{F\}$ , lo cual no es válido porque si la precondition se cumple, el programa termina, pero la postcondición no se cumple.
8. En el otro caso, si  $p_2$  es  $x = 1$ , entonces  $\{p_2\}S\{q_2\}$  es  $\{x = 1\}S\{F\}$ , lo cual no es válido por la misma razón que el caso anterior.
9. Por lo tanto  $\{p_1 \wedge p_2\}S\{q_1 \wedge q_2\}$  es válido, pero ni  $\{p_1\}S\{q_1\}$  ni  $\{p_2\}S\{q_2\}$  son válidos.
10. No vale lo afirmado.

## 4. Probar por medio del método H las fórmulas de correctitud parcial siguientes, relacionadas respectivamente a programas que calculan el valor absoluto de un número entero y el producto de dos números naturales:

a.  $\{x = X\} \text{if } x > 0 \text{ then } y := x \text{ else } y := -x \{y = |X|\}$ , siendo  $|X|$  el valor absoluto de  $X$ .

1. Se probará la terna de Hoare usando principalmente la regla del if.

2. Probar  $\{p \wedge B\}S_1\{q\}$ :

i.  $\{x = X \wedge x > 0\}y := x\{y = |X|\}$  (COND)

ii.  $\{x = |X|\}y := x\{y = |X|\}$  (ASI)

iii.  $x = X \wedge x > 0$

iv.  $\Rightarrow X > 0$

v.  $\Rightarrow X = |X|$

vi.  $\Rightarrow x = |X|$

vii. Conclusión:  $(x = X \wedge x > 0) \rightarrow (x = |X|)$

viii.  $\{x = X \wedge x > 0\}y := x\{y = |X|\}$  (CONS)

3. Probar  $\{p \wedge \neg B\}S_2\{q\}$ :

i.  $\{x = X \wedge \neg(x > 0)\}y := -x\{y = |X|\}$  (COND)

ii.  $\{x = X \wedge x \leq 0\}y := -x\{y = |X|\}$  (MAT)

iii.  $\{-x = |X|\}y := -x\{y = |X|\}$  (ASI)

iv.  $x = X \wedge \neg(x > 0)$

v.  $\Rightarrow x = X \wedge x \leq 0$

vi.  $\Rightarrow X \leq 0$

vii.  $\Rightarrow -X = |X|$

viii.  $\Rightarrow -x = |X|$

ix. Conclusión:  $(x = X \wedge \neg(x > 0)) \rightarrow (-x = |X|)$

x.  $\{x = X \wedge \neg(x > 0)\}y := -x\{y = |X|\}$  (CONS)

4. Finalmente:

- i.  $\{x = X \wedge x > 0\}y := x\{y = |X|\}$
- ii.  $\{x = X \wedge \neg(x > 0)\}y := -x\{y = |X|\}$
- iii.  $\{x = X\}\text{if } x > 0 \text{ then } y := x \text{ else } y := -x\{y = |X|\}$  (1, 2, COND)

b.  $\{x \geq 0 \wedge y \geq 0\}prod := 0; k := y; \text{while } k > 0 \text{ do } prod := prod + x; k := k - 1; \text{od}\{prod = x \cdot y\}$

1. Elementos:

- i. Precondición:  $p$  es  $(x \geq 0 \wedge y \geq 0)$
- ii. Postcondición:  $prod = x \cdot y$
- iii. La condición del while  $B$  es  $k > 0$

2. Se debe hallar un invariante  $i$  que sea verdadero antes de entrar al while, que sea verdadero en cada iteración del while, y que sea verdadero al salir del while. Analizando el programa se pueden notar varias cosas:

- i.  $y$  es la cantidad total de veces que queremos sumar  $x$ .
- ii.  $k$  es la cantidad de veces que aún falta sumar  $x$ . Siempre vale  $y$  al inicio del programa, y siempre vale 0 al finalizar el programa, porque justamente una vez vale 0 se sale del loop. Además, se puede ver claramente que  $k \geq 0$  siempre, porque empieza siendo igual a  $y$  que es mayor o igual a 0, y al salir del loop obviamente vale 0, por lo que se cumple trivialmente que  $k$  siempre es mayor o igual a 0.
- iii.  $y - k$  es la cantidad de veces que ya se sumó  $x$ .
- iv. Como en cada iteración del while se le suma  $x$  a  $prod$ , entonces  $prod$  es igual a  $x$  multiplicado por la cantidad de veces que ya se sumó  $x$ , es decir,  $prod = x \cdot (y - k)$ .
- v. Por lo tanto, un invariante posible es  $i \equiv prod = x \cdot (y - k) \wedge k \geq 0$ .

3. Probar  $p \rightarrow i$  para las primeras dos sentencias antes de entrar al while:  $\{x \geq 0 \wedge y \geq 0\}prod := 0; k := y; \{prod = x \cdot (y - k) \wedge k \geq 0\}$

- i.  $\{prod = x \cdot (y - y) \wedge y \geq 0\}k := y; \{prod = x \cdot (y - k) \wedge k \geq 0\}$  (ASI)
- ii.  $\{prod = 0 \wedge y \geq 0\}k := y; \{prod = x \cdot (y - k) \wedge k \geq 0\}$  (MAT)
- iii.  $\{0 = 0 \wedge y \geq 0\}prod := 0; \{prod = 0 \wedge y \geq 0\}$  (ASI)
- iv.  $\{y \geq 0\}prod := 0; \{prod = 0 \wedge y \geq 0\}$  (MAT)
- v.  $\{y \geq 0\}prod := 0; k := y; \{prod = x \cdot (y - k) \wedge k \geq 0\}$  (4, 2, SEC)

vi. Se reemplaza la precondición del paso 5 por otra precondición que la implique, por la regla CONS:  $(x \geq 0 \wedge y \geq 0) \rightarrow y \geq 0$  trivialmente verdadero porque  $(q \wedge p) \rightarrow p$  es una tautología.

vii.  $\{x \geq 0 \wedge y \geq 0\}prod := 0; k := y; \{prod = x \cdot (y - k) \wedge k \geq 0\}$  (CONS)

4. Probar  $i \wedge B \rightarrow i$ :  $\{prod = x \cdot (y - k) \wedge k \geq 0 \wedge k > 0\}prod := prod + x; k := k - 1; \{prod = x \cdot (y - k) \wedge k \geq 0\}$ .

- i.  $\{prod = x \cdot (y - (k - 1)) \wedge k - 1 \geq 0\}k := k - 1; \{prod = x \cdot (y - k) \wedge k \geq 0\}$  (ASI)
- ii.  $\{prod = x \cdot (y - k + 1) \wedge k - 1 \geq 0\}k := k - 1; \{prod = x \cdot (y - k) \wedge k \geq 0\}$  (MAT)
- iii.  $\{prod + x = x \cdot (y - k + 1) \wedge k - 1 \geq 0\}prod := prod + x; \{prod = x \cdot (y - k + 1) \wedge k - 1 \geq 0\}$  (ASI)
- iv.  $\{prod = x \cdot (y - k + 1) - x \wedge k - 1 \geq 0\}prod := prod + x; \{prod = x \cdot (y - k + 1) \wedge k - 1 \geq 0\}$  (MAT)
- v.  $\{prod = x \cdot (y - k + 1) - x \wedge k - 1 \geq 0\}prod := prod + x; k := k - 1; \{prod = x \cdot (y - k) \wedge k \geq 0\}$  (4, 2, SEC)

- vi.  $\{prod = x \cdot (y - k) \wedge k - 1 \geq 0\} prod := prod + x; k := k - 1; \{prod = x \cdot (y - k) \wedge k \geq 0\}$   
(MAT)
- vii. Se reemplaza la precondition del paso 6 por otra precondition que la implique, por la regla CONS:  $(prod = x \cdot (y - k) \wedge k \geq 0 \wedge k > 0) \rightarrow (prod = x \cdot (y - k) \wedge k - 1 \geq 0)$ :
  - a.  $k > 0 \Rightarrow k - 1 \geq 0$
  - b.  $(prod = x \cdot (y - k) \wedge k \geq 0 \wedge k > 0) \Rightarrow (prod = x \cdot (y - k) \wedge k - 1 \geq 0)$
- viii.  $\{prod = x \cdot (y - k) \wedge k \geq 0 \wedge k > 0\} prod := prod + x; k := k - 1; \{prod = x \cdot (y - k) \wedge k \geq 0\}$  (CONS)
- 5. Probar  $i \wedge \neg B \rightarrow prod = x \cdot y$ :  $(prod = x \cdot (y - k) \wedge k \geq 0 \wedge \neg(k > 0)) \rightarrow prod = x \cdot y$ .
  - i.  $(prod = x \cdot (y - k) \wedge k \geq 0 \wedge \neg(k > 0))$
  - ii.  $\Rightarrow (prod = x \cdot (y - k) \wedge k \geq 0 \wedge k \leq 0)$
  - iii.  $\Rightarrow (prod = x \cdot (y - k) \wedge k = 0)$
  - iv.  $\Rightarrow (prod = x \cdot y \wedge k = 0)$
  - v.  $\Rightarrow (prod = x \cdot y)$
  - vi. Conclusión:  $(prod = x \cdot (y - k) \wedge k \geq 0 \wedge \neg(k > 0)) \rightarrow prod = x \cdot y$ .
- 6. Finalmente encadenamos todo:
  - i.  $\{x \geq 0 \wedge y \geq 0\} prod := 0; k := y; \{i\}$  (probado en el paso 3)
  - ii.  $\{i \wedge k > 0\} prod := prod + x; k := k - 1; \{i\}$  (probado en el paso 4)
  - iii.  $\{i\} \text{while } k > 0 \text{ do } prod := prod + x; k := k - 1 \text{ od } \{i \wedge k \leq 0\}$  (REP)
  - iv.  $\{x \geq 0 \wedge y \geq 0\} prod := 0; k := y; \text{while } k > 0 \text{ do } prod := prod + x; k := k - 1 \text{ od } \{i \wedge k \leq 0\}$  (1, 3, SEC)
  - v.  $i \wedge k \leq 0 \rightarrow prod = x \cdot y$  (probado en el paso 5)
  - vi.  $\{x \geq 0 \wedge y \geq 0\} prod := 0; k := y; \text{while } k > 0 \text{ do } prod := prod + x; k := k - 1 \text{ od } \{prod = x \cdot y\}$  (4, 5, CONS)

**5. Se verificó en clase, usando el método H:  $\{x \geq 0 \wedge y > 0\} S_{div} :: q := 0; r := x; \text{while } r \geq y \text{ do } r := r - y; q := q + 1 \text{ od } \{x = q \cdot y + r \wedge 0 \leq r < y\}$  siendo  $S_{div}$  un programa que calcula por restas sucesivas la división entera de  $x$  sobre  $y$  en  $q$ , dejando el resto en  $r$ . Se pide ahora probar en  $H: \{x > 0 \wedge y = 0\} S_{div} \{false\}$  que significa que el programa  $S_{div}$  no termina a partir de la precondition  $(x > 0 \wedge y = 0)$ .**

...

**6. Probar la terminación del programa planteado en el ejercicio 4.b, es decir:  $\langle x \geq 0 \wedge y \geq 0 \rangle S_{prod} :: prod := 0; k := y; \text{while } k >$**

0 do  $prod := prod + x; k := k - 1$  od  $\langle true \rangle$ . Ayuda: Notar que  $k$  se decrementa en cada iteración y que se mantiene siempre mayor o igual que cero.

Para probar la correctitud total de una terna de Hoare, se debe probar la correctitud parcial y además que el programa termina. En el ejercicio 4.b ya se demostró la parcial, por lo que ahora solo falta probar la terminación.

Para probar la terminación, se debe hallar una función variante  $f$  que asegure que el número de iteraciones del bucle es finito, ergo que el programa termina. La función debe cumplir dos condiciones: a) decrecer en cada iteración del bucle b) tener un límite inferior  $\rightarrow$  no puede decrecer infinitamente.

Se observó en el ejercicio 4.b que  $k$  es la variable que controla cuándo el bucle termina, y además que siempre decrece en una unidad entera en cada iteración del bucle. Además, por el invariante que se encontró, se sabe que siempre  $k \geq 0$ . Por lo tanto, una función variante posible es  $f(k) = k$ , porque  $f$  decrece en cada iteración del bucle, y además tiene un límite inferior que es 0, ya que  $k$  siempre es mayor o igual a 0.

Verificamos las dos condiciones de la función variante  $f(k) = k$ :

1. Acotación inferior: como se sabe que  $k \geq 0$  siempre, entonces  $f(k) \geq 0$ , por lo que  $f$  tiene un límite inferior.
2. Decrecimiento: en cada iteración se ejecuta la instrucción  $k := k - 1$ . Como la función variante es  $f(k) = k$ , luego de una iteración se tiene  $f'(k) = k - 1$ . Como  $k - 1 < k$  para cualquier  $k$ , entonces  $f'(k) < f(k)$ , por lo que  $f$  decrece en cada iteración del bucle.

Como existe una función variante  $f$  que está acotada inferiormente y decrece estrictamente en cada iteración, el bucle ejecuta una cantidad finita de iteraciones. Por lo tanto, el programa termina.

Dado que en el ejercicio 4.b ya se probó la correctitud parcial y ahora se probó la terminación, se sigue que el programa es totalmente correcto.